

Tutorial - Semana 1, Encontro 1

Introdução

Toda game engine — Godot, Unity, Unreal ou qualquer outra — resolve o mesmo conjunto de problemas recorrentes de um jogo: renderizar imagens na tela, simular física, capturar input do jogador, organizar o mundo em cenas e gerenciar assets. Ela existe para que a equipe de desenvolvimento não precise reescrever essa camada a cada novo projeto e possa focar no jogo em si.

Este primeiro encontro da disciplina não trata de "como usar o Godot", mas de entender por que uma engine existe antes de tocar em qualquer botão. A partir daqui começa a construção do Vertical Slice único do semestre — o projeto de trabalho *O Templo Esquecido* —, que crescerá de forma incremental até a Semana 17. Nada do que for criado hoje será descartado.

Objetivos da semana

- Compreender o papel de uma game engine na produção de um jogo.
- Reconhecer as áreas principais do Godot Editor (Viewport, FileSystem Dock).
- Criar e organizar a estrutura inicial do projeto do Vertical Slice.

Resultado esperado ao final da semana

Ao final da Semana 1 (Encontros 1 e 2), cada estudante terá um projeto Godot 4.7 organizado, com uma primeira Scene funcional composta por Nodes filhos, relacionando o modelo de composição do Godot ao par GameObject/Component da Unity. Este tutorial cobre apenas o **Encontro 1**: a criação e organização do projeto.

Pré-requisitos

- Conhecimento prévio de desenvolvimento de jogos em Unity (a disciplina não ensina conceitos básicos de programação ou de game design).
 - Nenhum conhecimento prévio de Godot é exigido — esta é a primeira semana de contato com a engine.
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- Nenhum projeto prévio: esta é a primeira atividade prática da disciplina.

Arquivos necessários

- Nenhum arquivo de projeto prévio. O projeto será criado do zero neste encontro.

Assets utilizados

- Nenhum asset é necessário no Encontro 1. Os pacotes Kenney (Prototype Kit, Dungeon Kit, Nature Kit) começam a ser utilizados a partir da Semana 3.

Projeto esperado

- Godot 4.7 instalado.
- Addon Orchestrator instalado e habilitado (necessário a partir do Encontro 2, mas recomenda-se já confirmar a instalação neste encontro).

Imagem sugerida

Objetivo: mostrar a tela de download do Godot 4.7 no site oficial, destacando a versão correta (Standard, não .NET, já que a disciplina usa GDScript/Orchestrator). Enquadramento: captura de tela do navegador na página de downloads do Godot Engine. Elementos importantes: botão de download da versão 4.7, seletor de plataforma (Windows/Linux/macOS). Destaque: a versão exata "4.7" deve estar visível. Legenda sugerida: "Página oficial de download do Godot 4.7."

Parte 1 — O que é uma game engine e por que ela existe

Objetivo

Entender, antes de qualquer tela do Godot, qual problema uma game engine resolve e por que todas as engines (Godot, Unity, Unreal) compartilham a mesma finalidade central.

Conceito

Um jogo, por baixo da jogabilidade visível, precisa resolver repetidamente um conjunto fixo de problemas técnicos: como desenhar imagens na tela a cada frame (renderização), como simular colisões e movimento (física), como traduzir o toque no teclado ou controle em uma ação no jogo (input), como organizar os objetos de um nível (cena) e como carregar e gerenciar arquivos de arte, som e dados (assets).

Uma game engine é o software que resolve essa camada de forma reutilizável, para que cada novo jogo não precise reescrever renderização, física ou gerenciamento de assets do zero. É exatamente o mesmo papel que a Unity cumpre — a diferença entre engines está em *como* cada uma organiza essas soluções, não em se elas existem.

Essa distinção entre "o que é universal" e "como o Godot implementa" é o primeiro dos quatro pontos que todo conteúdo da disciplina deve responder, e será retomada a cada novo sistema apresentado ao longo do semestre.

Passo a passo

1. Antes de abrir o Godot, discuta em dupla ou com a turma: o que a Unity resolveu "por trás" dos GameObjects nos projetos que vocês já fizeram?
2. Liste, em voz alta, os problemas que uma engine resolve: renderização, física, input, organização de cena, assets.
3. Abra o Godot Engine (não crie o projeto ainda) e observe a tela inicial de gerenciamento de projetos ("Project Manager").
4. Note que o Project Manager já é, por si só, uma ferramenta de organização — cada projeto Godot é isolado dos demais.

Resultado esperado

A turma consegue nomear, sem depender de sintaxe do Godot, os problemas centrais que qualquer engine resolve, e reconhece que esses problemas já foram enfrentados anteriormente ao trabalhar com Unity.

Verificando

Peça a um colega para explicar, com as próprias palavras, por que uma engine existe. Se a resposta mencionar apenas "fazer jogos mais fácil" sem citar nenhum dos problemas concretos (renderização, física, input, cena, assets), retome a explicação.

Problemas comuns

- Confundir "engine" com "editor" — o editor é a interface visual para trabalhar com a engine, mas a engine é o conjunto de sistemas por trás dela (renderização, física etc.).
- Achar que a resposta é específica do Godot — reforçar que a resposta vale para qualquer engine.

Boas práticas

- Sempre que uma nova ferramenta do Godot for apresentada ao longo do semestre, perguntar primeiro "que problema universal isso resolve?" antes de perguntar "como se usa no Godot?".

Comparação com Unity

A Unity resolve exatamente os mesmos problemas (renderização, física, input, cena, assets), com uma organização de projeto equivalente: uma janela de gerenciamento de projetos (Unity Hub) antes de abrir o editor propriamente dito, assim como o Project Manager do Godot.

Parte 2 — Tour guiado pelo Godot Editor

Objetivo

Reconhecer as áreas principais do Godot Editor — em especial o Viewport e o FileSystem Dock — e seu papel dentro do fluxo de trabalho.

Conceito

O Godot Editor organiza o trabalho em painéis (docks) especializados. Dois deles são centrais desde o primeiro contato:

- O **Viewport** é a janela onde a cena é visualizada e editada espacialmente — o equivalente à Scene View da Unity.
- O **FileSystem Dock** é o painel que lista todos os arquivos e pastas do projeto (scripts, cenas, assets, recursos) — o equivalente ao Project window (painel de Assets) da Unity.

Assim como a Unity, o Godot separa "o que existe no projeto" (FileSystem Dock / Project window) de "o que está sendo editado espacialmente agora" (Viewport / Scene View). Entender essa separação evita que o estudante confunda arquivos do projeto com objetos de uma cena específica.

Passo a passo

1. No Project Manager do Godot, clique em **New Project**.
2. Dê um nome ao projeto (ex.: `TemploEsquecido`) e escolha uma pasta vazia no disco.
3. Em **Renderer**, mantenha o padrão (Forward+) recomendado para projetos 3D.
4. Clique em **Create & Edit** para abrir o projeto no editor.
5. Observe a barra superior central, onde ficam as abas **2D**, **3D**, **Script** e **AssetLib** — essas abas alternam o modo de trabalho do editor.
6. Clique na aba **3D** e observe o **Viewport** — a área central onde a cena 3D é exibida.
7. Localize o **FileSystem Dock**, geralmente no canto inferior esquerdo, e observe que ele já lista a pasta raiz do projeto (`res://`).
8. Expanda e explore a estrutura padrão criada pelo Godot (arquivo `project.godot`, ícone do motor).

Resultado esperado

O estudante consegue apontar, na tela do editor, onde fica o Viewport e onde fica o FileSystem Dock, e explicar a função de cada um sem confundi-los.

Verificando

Peça ao estudante para localizar um arquivo no FileSystem Dock e depois localizar a área onde uma cena 3D seria exibida (Viewport), sem ajuda do professor.

Problemas comuns

- Confundir o Viewport com o "jogo em execução" — reforçar que o Viewport é a área de edição, não o jogo rodando (isso só aparece ao pressionar Play, mais adiante no semestre).
- Não encontrar o FileSystem Dock porque o layout de painéis foi alterado acidentalmente — usar o menu **Editor > Editor Layout > Default** para restaurar.

Boas práticas

- Nunca mover ou renomear arquivos pelo explorador de arquivos do sistema operacional — sempre usar o FileSystem Dock do Godot, que atualiza automaticamente as referências internas do projeto.

Comparação com Unity

O FileSystem Dock do Godot cumpre o mesmo papel do painel **Project** da Unity: listar e organizar os assets do projeto. O Viewport do Godot equivale à **Scene View** da Unity. A diferença mais relevante aparecerá no Encontro 2: no Godot, uma Scene é uma árvore de Nodes salva em um único arquivo `.tscn`, enquanto na Unity uma Scene é um contêiner de GameObjects.

Parte 3 — Criação e organização da estrutura inicial do projeto

Objetivo

Estruturar as pastas do projeto do Vertical Slice desde o primeiro dia, seguindo a convenção oficial adotada na disciplina (ver `PROJECT_ARCHITECTURE.md`).

Conceito

Um projeto de jogo cresce rapidamente em número de arquivos. Organizar a estrutura de pastas desde o início evita retrabalho nas próximas 16 semanas, já que cada sistema futuro (Player, Interactables, UI, Autoloads, Resources) terá um local fixo e previsível. Essa organização é uma convenção de projeto, não uma exigência técnica do Godot — mas é a convenção que será seguida por toda a disciplina.

Passo a passo

1. No FileSystem Dock, clique com o botão direito na pasta raiz (`res://`).
2. Selecione **New Folder** e crie a pasta `scenes` .
3. Dentro de `scenes` , crie as subpastas `characters` , `interactables` , `ui` e `levels` .
4. Dentro de `levels` , crie as subpastas `exploration` e `dungeon` .
5. Na raiz do projeto, crie a pasta `scripts` e, dentro dela, as subpastas `autoload` e `components` .
6. Crie a pasta `orchestrations` na raiz (para os arquivos `.torch` do Orchestrator).
7. Crie a pasta `resources` na raiz e, dentro dela, as subpastas `items` e `save` .
8. Crie as pastas `assets` (com subpastas `dungeon` e `nature`), `materials` , `audio` e `animations` na raiz.
9. Confira a estrutura final no FileSystem Dock, comparando com a tabela de organização do `PROJECT_ARCHITECTURE.md`.

Resultado esperado

O projeto possui a estrutura de pastas completa definida no `PROJECT_ARCHITECTURE.md`, mesmo que a maioria delas esteja vazia — elas serão preenchidas ao longo do semestre.

Verificando

Compare a estrutura de pastas criada com a listagem abaixo. Nenhuma pasta deve estar faltando, mesmo que vazia:

```
res://
├─ scenes/
│   ├─ characters/
│   ├─ interactables/
│   ├─ ui/
│   └─ levels/
```

```
|      |─ exploration/
|      |─ dungeon/
|─ scripts/
|  |─ autoload/
|  |─ components/
|─ orchestrations/
|─ resources/
|  |─ items/
|  |─ save/
|─ assets/
|  |─ dungeon/
|  |─ nature/
|─ materials/
|─ audio/
|─ animations/
```

Problemas comuns

- Criar pastas com nomes em PascalCase ou com espaços — a convenção da disciplina usa snake_case para pastas e arquivos.
- Esquecer subpastas (ex.: criar `levels` mas esquecer `exploration / dungeon`) — revisar contra a lista acima antes de encerrar o encontro.

Boas práticas

- Nenhum asset deve ficar solto na raiz de `res://` — todo arquivo pertence a uma subpasta temática, regra que vale para o restante do semestre (ver PROJECT_ARCHITECTURE.md, seção 9).
- Criar a estrutura completa agora, mesmo vazia, evita decisões de organização apressadas em semanas futuras, quando o foco estará em outros conceitos.

Comparação com Unity

Na Unity, a convenção mais comum é organizar o painel Project em pastas como `Scripts`, `Scenes`, `Prefabs` e `Materials` dentro de `Assets/`. O princípio é o mesmo — organização temática por tipo/sistema —, mas o Godot não usa uma pasta `Assets` obrigatória: qualquer subpasta de `res://` já é reconhecida automaticamente pelo motor.

Ao final da semana

Este tutorial cobre apenas o Encontro 1. Ao final da Semana 1 completa (Encontros 1 e 2), o projeto deverá conter a estrutura de pastas criada aqui, mais uma primeira Scene funcional com Nodes filhos (construída no Encontro 2). Segundo o PROJECT_ARCHITECTURE.md (seção 6, Módulo 1), este encontro corresponde ao item "Estrutura inicial do projeto", pré-requisito para todos os sistemas do Módulo 1.

Desafio

Organize uma pasta adicional dentro da estrutura do projeto (por exemplo, uma pasta `prototypes` ou `references`) que ainda não foi sugerida neste tutorial, e justifique brevemente a um colega por que ela é útil para o Vertical Slice. Não há solução única — o objetivo é praticar organização própria dentro da convenção compartilhada.

Checklist

- ☐ Godot 4.7 instalado e projeto criado com sucesso
- ☐ Projeto abre sem erros
- ☐ Estrutura de pastas completa (scenes, scripts, orchestrations, resources, assets, materials, audio, animations, com todas as subpastas)
- ☐ Nenhum arquivo solto na raiz de `res://`
- ☐ Pasta adicional do desafio criada e justificada

Glossário

- **Game Engine:** software que resolve, de forma reutilizável, os problemas recorrentes de um jogo (renderização, física, input, cena, assets).
- **Viewport:** área do editor onde uma cena é visualizada e editada espacialmente.
- **FileSystem Dock:** painel do Godot que lista todos os arquivos e pastas do projeto.
- **res://** caminho raiz que representa a pasta do projeto Godot atual.
- **Project Manager:** tela inicial do Godot para criar, abrir e gerenciar projetos.

Referências

- Godot Documentation — Getting Started / Introduction:
https://docs.godotengine.org/en/stable/getting_started/introduction/index.html
- Godot Documentation — Nodes and Scenes:
https://docs.godotengine.org/en/stable/tutorials/scripting/nodes_and_scene_instances.html
- GDQuest: <https://www.gdquest.com/>
- Unity Manual (consulta comparativa): <https://docs.unity3d.com/Manual/>